PostgreSQL: блокировки по числам

Клиент передаёт целое число n от 0 до N. При успехе сервер отвечает n + 1. Если n уже поступало или n + 1 уже успешно обработано, возвращается ошибка и создаётся запись в журнале. Проверка повтора выполняется первой.

Первое состояние каждого числа

Класс NumberRepository отделяет HTTP от хранения данных. В number_state записывается первый исход для числа: принято или отклонено. Любая существующая строка означает повтор. Перед чтением берутся транзакционные advisory-блокировки для n и n + 1 в возрастающем порядке. Соседние запросы ждут друг друга, а независимые числа могут обрабатываться параллельно.

Данные

Объект	Назначение
number_state	Число и признак принятия первого запроса
request_history	Все корректные обращения и их исходы
processed_numbers	SQL-представление принятых состояний

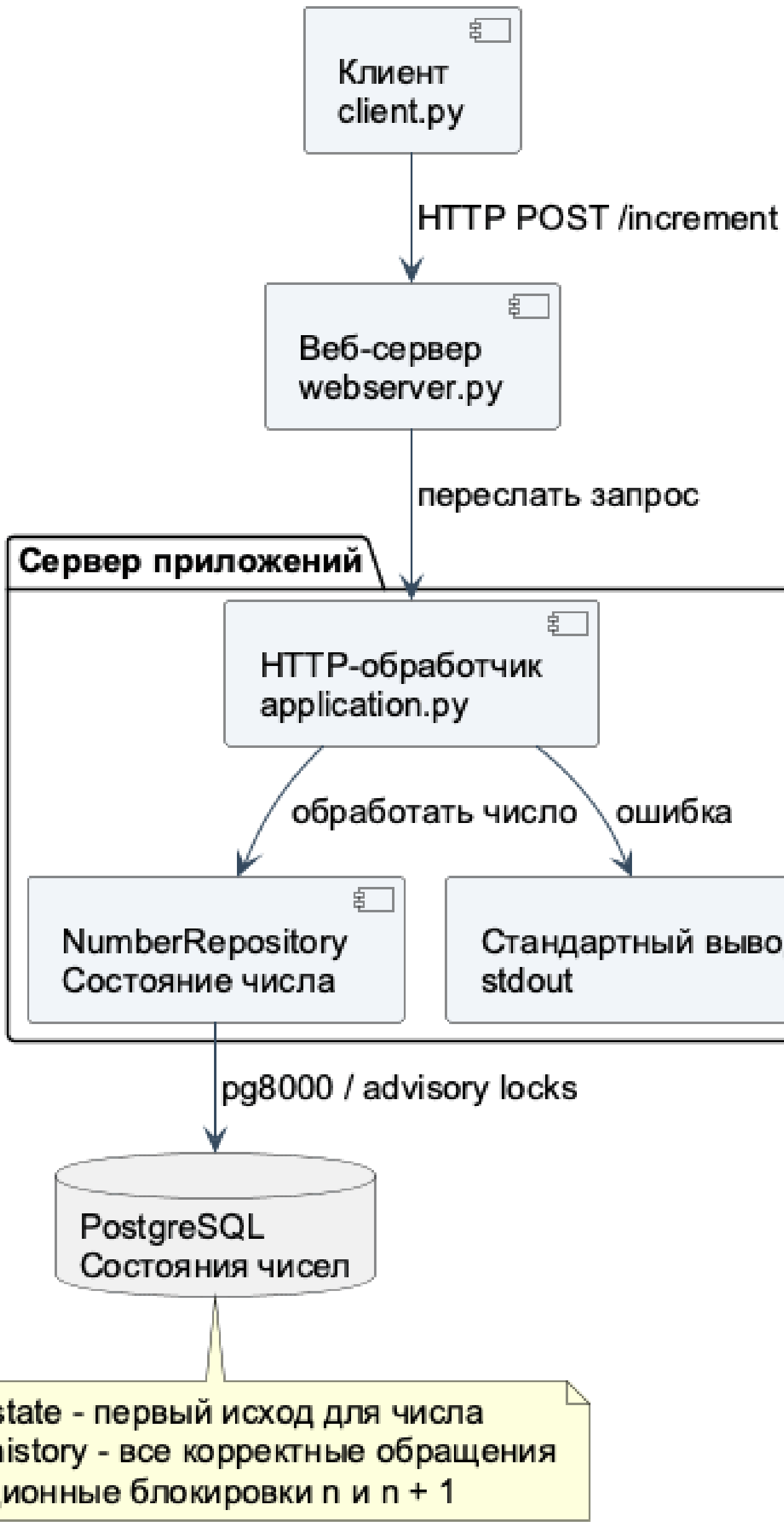
Ключевой фрагмент программы

```
for key in (number, number + 1):
    cur.execute(
        'SELECT pg_advisory_xact_lock(CAST(%s AS bigint))',
        (key,)
    )
```

Это фрагмент repository.py. Полная программа также проверяет ввод, обрабатывает ошибки и отменяет незавершённую транзакцию. В базе сохраняется вход n, а не вычисленный ответ n + 1.

Диаграмма компонентов

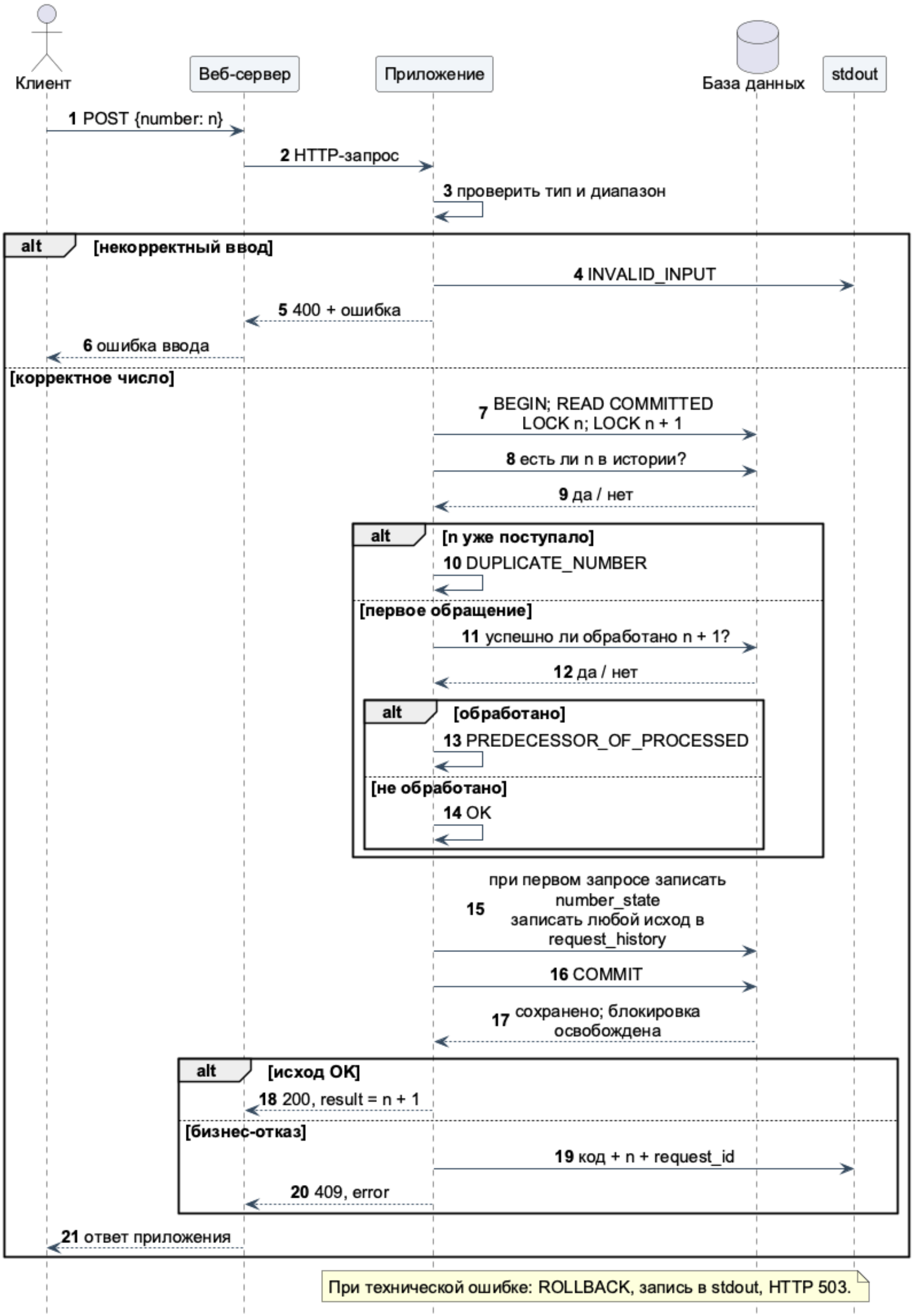
Клиент, веб-сервер и приложение выполняют разные роли. Веб-сервер только пересылает запрос. Приложение выбирает исход, база хранит историю. Представление, прикладная логика и данные образуют три логических уровня.



PostgreSQL работает отдельным сервером. Класс NumberRepository использует общие для всех процессов блокировки ключей. Журнал ошибок направлен в stdout приложения.

Диаграмма последовательностей

Время идёт сверху вниз. Ветви alt показывают разные результаты запроса. Блокировка удерживается до фиксации или отмены транзакции.



Корректное число с бизнес-отказом попадает в историю: повтор такого числа тоже считается повтором. Неверный JSON, тип или диапазон даёт ошибку ввода и не добавляет число в историю.

Примеры, проверка и запуск

Запросы по порядку	Ответ
5	200, результат 6
5 ещё раз	409, DUPLICATE_NUMBER
4	409, PREDECESSOR_OF_PROCESSED
6	200, результат 7

Все 11 интеграционных тестов прошли. В тестах использовались два процесса приложения и два веб-сервера с настоящей PostgreSQL. Из 20 одновременных одинаковых запросов ровно один завершился успешно, остальные получили ошибку повтора.

Проверены 15 некорректных входов, границы диапазона, 10 пар соседних чисел, сохранение истории после перезапуска, откат записи и недоступность сервисов. Обе бизнес-ошибки проверены в stdout и отсутствуют в stderr. run_local.py проверен отдельно: stdout приложения виден через запускатель.

Запуск из каталога варианта

```
python3 -m pip install -r requirements.txt
python3 run_local.py --pg-bin /path/to/postgresql/bin
# In another terminal:
python3 client.py 5
# Integration tests:
python3 test_integration.py --pg-bin /path/to/postgresql/bin
```

Python 3.12+. Нужны pg8000 и установленный PostgreSQL; проверено с PostgreSQL 17.6. Данные сохраняются в .runtime; Ctrl-C останавливает процессы. Порты и каталог можно изменить аргументами. Это локальный учебный сервис без авторизации и TLS.

Результаты: test_results.json и launch_results.json. Полные команды, формат HTTP и имена файлов: README.txt. Диаграммы доступны также в редактируемом .puml и в SVG.

Условие: Achievement #2, слайд 17 второй лекции. Код, схема БД и формат HTTP разработаны для варианта. Технический источник: PostgreSQL: advisory locks.